# Implementation of FastSpeech 2-Based Text-to-Speech Synthesis for English and Telugu

**Technical Report**

Submitted by

**Savio Shaju**

Department of Computer Science and Engineering

Saintgits College of Engineering

**GitHub Repository**

https://github.com/savioshaju/TTS.git

**Model Checkpoints and Audio Artifacts (Google Drive)**

https://drive.google.com/drive/folders/1MYS5zcsyehxDN5n4XaKj5jCidpUZ-efw?usp=sharing

# Abstract

This report describes how I implemented, trained, and evaluated two independent FastSpeech 2 text-to-speech (TTS) systems, one for Indian English and one for Telugu. FastSpeech 2 is a non-autoregressive model: it predicts the whole mel-spectrogram in parallel from phoneme or character embeddings, guided by variance predictors for duration, pitch ($F_0$), and energy. This avoids the exposure bias and the slow, step-by-step inference of autoregressive models.

Both systems were built under an assignment constraint of roughly 5 hours of training audio and 1 hour of validation audio. The English system uses crowdsourced multi-speaker recordings from the AI4Bharat Svarah corpus, aligned with the Montreal Forced Aligner (MFA) over 100 IPA phoneme tokens. The Telugu system uses studio recordings from the IndicTTS corpus, aligned directly with Meta's Massively Multilingual Speech (MMS) Wav2Vec2-CTC model over 460 native-script akshara tokens. Each acoustic model was trained for 100,000 steps with mixed precision on two NVIDIA Tesla T4 GPUs and paired with a pre-trained HiFi-GAN vocoder, which I adapted to the 16 kHz setting through sinc resampling and a duration-scale calibration.

I evaluated each system on five prescribed test sentences. The outputs show clear articulation of long words, consonant conjuncts, heteronyms, and spoken dates, with plausible pitch and energy variation, and the systems handle the intentional spacing typos and a loanword included in the test set.

# Contents

**Abstract** **i**

**1 Introduction** **1**
1.1 Background and Problem Statement . . . 1
1.2 FastSpeech 2 Architecture Principles . . . 1
1.3 Summary of Key Results . . . 1
1.4 Report Organization . . . 2

**2 Implementation Details** **3**
2.1 Base Architecture and Dependencies . . . 3
2.2 Codebase Structure . . . 3
2.3 Codebase Corrections and Bug Fixes . . . 3
2.4 Language-Specific Modifications . . . 4

**3 Datasets and Preprocessing** **5**
3.1 Dataset Selection . . . 5
3.2 Audio Preprocessing . . . 5
3.3 Train/Validation Split . . . 5
3.4 Text Processing and Tokenization . . . 6
3.4.1 English: Phoneme Tokenization . . . 6
3.4.2 Telugu: Akshara Tokenization . . . 6
3.5 Forced Alignment . . . 7
3.5.1 English: Montreal Forced Aligner . . . 7
3.5.2 Telugu: Meta MMS Alignment . . . 7
3.6 Acoustic Features . . . 7
3.6.1 Mel-Spectrograms . . . 7
3.6.2 Pitch ($F_0$) . . . 7
3.6.3 Energy . . . 7
3.6.4 Feature Normalization . . . 8

**4 Model Architecture and Vocoder** **9**
4.1 FastSpeech 2 Architecture . . . 9
4.2 Loss Functions . . . 9
4.3 HiFi-GAN Vocoder Adaptation . . . 10

**5 Training and Loss Analysis** **11**
5.1 Training Hyperparameters . . . 11
5.2 Loss Curves . . . 11
5.3 Convergence and Checkpoints . . . 11
5.4 Why Validation Loss Is Higher Than Training Loss . . . 11

**6 Speech Synthesis and Test Sentence Evaluation** **15**
6.1 Synthesis Setup . . . 15
6.2 Acoustic Measurements . . . 15
6.3 English Test Sentences . . . 15
6.3.1 Sentence E1: Long Word Articulation . . . 15
6.3.2 Sentences E2a & E2b: Heteronym Stress . . . 16
6.3.3 Sentence E3: Typographical Error and Split-Word Handling . . . 18

6.3.4 Sentence E4: Dates, Numbers, and Loanwords . . . . . 18
6.4 Telugu Test Sentences . . . . . 19
6.4.1 Sentence T1: Consonant Conjuncts . . . . . 19
6.4.2 Sentences T2a & T2b: Contextual Word Stress . . . . . 19
6.4.3 Sentence T3: Space Inside a Word . . . . . 21
6.4.4 Sentence T4: Long Compound Sentence . . . . . 21

**7 Comparison: English vs. Telugu** **23**
7.1 Linguistic and Acoustic Differences . . . . . 23
7.2 Limitations and Failure Cases . . . . . 23
7.3 Potential Improvements . . . . . 23

**8 Reproducibility** **24**
8.1 Environment and Dependencies . . . . . 24
8.2 Dataset Download and Directory Setup . . . . . 24
8.3 Preparing the 5-Hour Datasets . . . . . 24
8.4 Audio Standardization, Forced Alignment, and Feature Extraction . . . . . 25
8.5 Model Training: The Kaggle Dual-GPU Method . . . . . 26
8.6 Model Weights and Checkpoints . . . . . 27
8.7 Inference and Evaluation Commands . . . . . 27
8.7.1 Speaker Selection and CLI Control . . . . . 27

**9 Conclusion** **28**

# 1. Introduction

## 1.1 Background and Problem Statement

Text-to-Speech (TTS) synthesis turns written text into a spoken waveform. Most neural TTS systems work in two stages: an acoustic model first predicts an intermediate time-frequency representation (here, an 80-channel log mel-spectrogram), and a neural vocoder then converts it into audio. Autoregressive acoustic models such as Tacotron 2 can sound expressive, but they generate frames one after another. That makes inference slow, lets errors accumulate over time, and can cause attention failures that repeat or skip phonemes, especially on text unlike the training data.

Non-autoregressive models, and FastSpeech 2 [1] in particular, avoid these problems by predicting the entire mel-spectrogram in parallel with Feed-Forward Transformer (FFT) blocks. Instead of learning an attention alignment, FastSpeech 2 models timing explicitly: a duration predictor is trained on durations from a forced aligner. A variance adaptor then adds explicit pitch ($F_0$) and energy information to the hidden representation.

The goal of this assignment was to adapt, train, and evaluate two independent FastSpeech 2 acoustic models, each using approximately 5 hours of training audio and 1 hour of validation audio:

1. **Indian English:** multi-speaker audio from the AI4Bharat Svarah corpus, which covers a range of regional Indian accents, with an IPA phoneme front-end.

2. **Telugu:** studio recordings from the IndicTTS corpus, using native-script akshara tokens. Because Telugu spelling is largely phonetic, this removes the need for an external G2P lexicon.

The main engineering tasks were: building alignment pipelines (the Montreal Forced Aligner for English and Meta MMS CTC alignment for Telugu); decoupling the pitch and energy predictors; fixing device placement for multi-GPU training in PyTorch; adapting a 22.05 kHz HiFi-GAN universal vocoder to 16 kHz acoustic predictions through polyphase sinc resampling; and calibrating speaking pace.

## 1.2 FastSpeech 2 Architecture Principles

FastSpeech 2 rests on three ideas:

1. **Parallel generation.** Feed-forward transformer blocks replace recurrent, step-by-step decoding, so all mel-spectrogram frames are produced at once. This gives a large inference speedup (over $30\times$) compared with autoregressive baselines.

2. **Explicit length regulation.** Phoneme durations come from a forced aligner during training and from a convolutional duration predictor at inference. The length regulator repeats each phoneme embedding to match its duration in frames, so no attention alignment can collapse.

3. **Multi-task variance adaptation.** The same text can be spoken with many valid pitch and loudness patterns (the one-to-many problem in TTS). Pitch and energy predictors give the model explicit frame-level prosodic features to work with.

## 1.3 Summary of Key Results

- **Speech quality and intelligibility.** Both models produce clear articulation on all ten benchmark sentences. The English model renders the 28-letter word *antidisestablishmentarianism*, shows a stress shift between the noun and verb uses of *conduct*, handles the split token in *rhyth m*, and reads out dates, times, and the Telugu loanword *andaroo*. The Telugu model handles complex consonant conjuncts (న్యూరోఫార్మకోలాజికల్), gives the word పట్టు different pitch and energy in two different contexts, and inserts a clear pause between clauses in a long compound sentence.

- **Convergence and checkpoint choice.** Training loss fell steadily over 100,000 steps (English: 2.59 at step 11,000 to 0.84 at step 61,000; Telugu: 0.88 at step 30,000 to 0.64 at step 74,000). Validation loss reached its minimum early (2.5505 at step 11,000 for English; 2.1309 at step 30,000 for Telugu) and then flattened. In my listening, the minimum-validation-loss checkpoints sounded muffled, which is consistent with the smoothing effect of the L1 loss. Later checkpoints (step 61,000 for English, step 74,000 for Telugu) had noticeably sharper consonants, so I used those for synthesis.
- **Vocoder calibration and pacing.** Both acoustic models predict 16 kHz mel-spectrograms, while the HiFi-GAN universal vocoder was trained on 22.05 kHz audio. Sinc resampling from 22.05 kHz to 16 kHz restores the pitch and formant scale, but it shortens the audio by a factor of $\alpha \approx 0.7256$, so speech plays about $1.38\times$ too fast. Setting the inference duration control to $d_{\text{control}} = 1.25$ compensates for this and provides natural conversational pacing; the resulting speaking rates are listed in Table 6.1.
- **Dataset splits.** English was split into 4.98 hours (3,419 utterances) for training and 1.09 hours (746 utterances) for validation, across 104 speakers, with the split made within each speaker. Telugu was split into 5.00 hours (2,248 aligned utterances, from 2,250 raw) for training and 1.00 hour (445 utterances) for validation, with an exact 50/50 gender balance across the two studio speakers.

## 1.4 Report Organization

The rest of this report is organized as follows:

- **Chapter 2 (Implementation Details):** codebase structure, the PyTorch 2.5 multi-GPU setup, bug fixes, and language-specific modifications.
- **Chapter 3 (Datasets and Preprocessing):** corpus selection, audio filtering, train/validation splits, tokenization, forced alignment, and acoustic features.
- **Chapter 4 (Model Architecture and Vocoder):** encoder and decoder blocks, the variance adaptor, the loss function, and the HiFi-GAN vocoder adaptation.
- **Chapter 5 (Training and Loss Analysis):** hyperparameters, loss curves, convergence, and checkpoint selection.
- **Chapter 6 (Speech Synthesis and Evaluation):** acoustic measurements and variance-adaptor plots for all ten test sentences.
- **Chapter 7 (Comparison: English vs. Telugu):** phonetic and prosodic differences, limitations, and possible extensions.
- **Chapter 8 (Reproducibility):** dependencies, dataset download, preprocessing commands, Kaggle dual-GPU training, and inference instructions.
- **Chapter 9 (Conclusion):** main outcomes and lessons learned.

# 2. Implementation Details

## 2.1 Base Architecture and Dependencies

I started from an open-source PyTorch reference implementation of FastSpeech 2 that was originally built for single-speaker LJSpeech. Development and testing used Python 3.11 with PyTorch 2.5, TorchAudio, NumPy, SciPy, Librosa, PyWorld, and Matplotlib. Training ran on two NVIDIA Tesla T4 GPUs (16 GB VRAM each).

## 2.2 Codebase Structure

The English and Telugu pipelines live in separate, self-contained directories, each with its own preprocessing, training, configuration, and inference scripts:

```
project/
|-- English/
|   |-- train.py
|   |-- preprocess.py
|   |-- inference.py
|   |-- config.yaml
|   `-- checkpoints/
|-- Telugu/
|   |-- train.py
|   |-- preprocess.py
|   |-- inference.py
|   |-- config.yaml
|   `-- checkpoints/
|-- Results/
|   |-- English/
|   |   |-- training_loss.png
|   |   |-- validation_loss.png
|   |   |-- test_generated_audios/
|   |   `-- variance_adaptor_plots/
|   |       |-- duration/
|   |       |-- pitch/
|   |       `-- energy/
|   `-- Telugu/
|       |-- training_loss.png
|       |-- validation_loss.png
|       |-- test_generated_audios/
|       `-- variance_adaptor_plots/
|           |-- duration/
|           |-- pitch/
|           `-- energy/
|-- Report/
|   `-- Technical_Report.pdf
|-- README.md
`-- requirements.txt
```

## 2.3 Codebase Corrections and Bug Fixes

While testing the reference implementation, I found and fixed three bugs:

1. **Energy control (`model/modules.py`).** In the variance adaptor, `p_control` was passed to the energy lookup instead of `e_control`. As a result, changing the pitch control also changed the energy, and the energy control had no effect. I corrected the arguments so that the two are independent.

2. **Multi-GPU device placement (`utils/tools.py`).** Tensors were created on `"cuda"` without a device index, which caused device mismatches under `DataParallel`. I changed the allocations to follow the device of the input batch.

3. **Text cleaner fallback (`text/__init__.py`).** When the input string was not wrapped in braces `{...}`, the cleaner silently fell back to raw ASCII character indices. The model was trained on phoneme IDs, so this produced mismatched indices and noisy output. I added an explicit validation check.

## 2.4 Language-Specific Modifications

Table 2.1 compares the reference FastSpeech 2 components with the changes I made for English and Telugu.

Table 2.1: Reference components versus modifications per language.

| **Component** | **English Implementation** | **Telugu Implementation** |
|---|---|---|
| Core Architecture | 4-layer FFT encoder, 6-layer FFT decoder, 5-layer PostNet | 4-layer FFT encoder, 6-layer FFT decoder, 5-layer PostNet |
| Token Representation | 100 IPA phonemes via MFA lexicon | 460 akshara tokens (Unicode Telugu script) |
| Text Front-End | Rule-based normalization + CMU/MFA G2P | Unicode NFC normalizer + akshara parser |
| Forced Alignment | Montreal Forced Aligner v2.2 (`english_mfa`) | Meta MMS CTC forced aligner (`mms-1b-all`) |
| Speaker Modeling | Multi-speaker: 104 learned embeddings ($104 \times 256$) | 2 studio speakers: learned embeddings ($2 \times 256$) |
| Audio Standard | 16 kHz mono WAV (Svarah corpus) | 16 kHz mono WAV (male 48 kHz resampled to 16 kHz, female 16 kHz native) |
| Vocoder Pipeline | HiFi-GAN universal + Sinc Resampler ($22.05 \rightarrow 16$ kHz) | HiFi-GAN universal + Sinc Resampler ($22.05 \rightarrow 16$ kHz) |
| Duration Calibration | $d_{\text{control}} = 1.25$ (calibrated conversational pacing) | $d_{\text{control}} = 1.25$ (calibrated conversational pacing) |

# 3. Datasets and Preprocessing

## 3.1 Dataset Selection

**English (AI4Bharat Svarah).** Svarah is an open-source, crowdsourced speech corpus that captures Indian English accents from speakers with many different linguistic backgrounds. One practical point: the upstream Hugging Face repository (`ai4bharat/Svarah`) provides only `data/test-*.parquet` shards (9.61 hours in total) and has no official `train` or `validation` partition. To meet the assignment's target of about 5 hours for training and 1 hour for validation, I extracted the 9.61-hour pool and sampled roughly 40 utterances from each of the 104 Indian speakers (54 female, 50 male), who cover 19 primary mother tongues (Hindi, Telugu, Tamil, Kannada, Marathi, Bengali, Odia, Gujarati, Malayalam, Urdu, and others). This gave 4,202 raw utterances. After alignment checks and audio quality filtering, **4,165 utterances (6.07 hours)** remained.

**Telugu (IndicTTS).** The IndicTTS Telugu corpus, curated by IIT Madras and MeitY, contains clean studio speech. The male recordings are at 48 kHz (1,433 files in the selected set) and the female recordings are at 16 kHz (1,262 files). To meet the 5-hour training and 1-hour validation targets, I built a 50/50 gender-balanced stratified partition (`telugu_prepared_v2`): 2,250 training utterances (5.0018 hours: 2.50 hours female, 2.50 hours male) and 445 validation utterances (1.0030 hours: 0.50 hours female, 0.50 hours male). Two training files could not be aligned during feature extraction and were dropped, leaving **2,248 aligned training utterances (5.00 hours)** and **445 validation utterances (1.00 hour)**.

## 3.2 Audio Preprocessing

The two corpora differ in recording setup and format, so I applied the following steps:

- **Resampling.** The 48 kHz Telugu male studio audio (1,433 files) was downsampled to 16 kHz with a 64-tap Kaiser-windowed polyphase sinc filter and an anti-aliasing cutoff of 7,600 Hz. The Telugu female audio (1,262 files) was already at 16 kHz. English audio was converted to 16 kHz, 16-bit mono PCM.
- **Duration and quality filtering.** Clips shorter than 1.0 second or longer than 12.0 seconds (14.5 seconds for Telugu) were removed. Very short clips carry too little prosodic context, and very long clips cause out-of-memory errors in self-attention. Clips with clipping or an SNR below 15 dB were also discarded.
- **Peak normalization.** All audio was peak-normalized to a maximum amplitude of 0.95.

## 3.3 Train/Validation Split

**English split.** The English model uses learned speaker embeddings ($104 \times 256$). If I had split by speaker, the embeddings of the held-out speakers would never have been trained, and the validation loss would have been misleading. I therefore split *within each speaker* (seed 42, roughly 82% train / 18% validation). Every speaker appears in both sets, and no utterance appears in both:

- **Training:** 3,419 utterances, 4.98 hours (17,928 s).
- **Validation:** 746 utterances, 1.09 hours (3,924 s).

**Telugu split.** The IndicTTS corpus was partitioned with an exact 50/50 gender-balanced duration stratification (`telugu_prepared_v2`). There is no overlap in utterances or transcripts between the two sets, which I verified with SHA-256 hashes of the text:

- **Training:** 2,248 aligned utterances (2,250 raw), **5.00 hours** (18,007 s), with 2.50 hours female (1,053 utterances) and 2.50 hours male (1,197 utterances).

- **Validation:** 445 utterances, **1.00 hour** (3,611 s), with 0.50 hours female (209 utterances) and 0.50 hours male (236 utterances).

Both splits meet the assignment's targets of about 5 hours for training and 1 hour for validation.

Table 3.1: Dataset statistics and acoustic feature parameters.

| **Parameter** | **English (Svarah)** | **Telugu (IndicTTS)** |
|---|---|---|
| Recording Environment | Crowdsourced, varied devices | Studio recordings |
| Native Sampling Rate | 16 kHz | 16 kHz (female) / 48 kHz (male) |
| Standardized Sampling Rate | 16 kHz mono | 16 kHz mono |
| Total Utterances / Duration | 4,165 utterances / 6.07 hours | 2,693 utterances / 6.00 hours |
| Mean Utterance Length | 5.25 seconds | 8.02 seconds |
| Speaker Count | 104 (54 Female / 50 Male) | 2 (1 Female / 1 Male) |
| Training Set | 3,419 utterances / 4.98 hours | 2,248 utterances / 5.00 hours |
| Validation Set | 746 utterances / 1.09 hours | 445 utterances / 1.00 hour |
| Test Sentences | 5 sentences (E1, E2a, E2b, E3, E4) | 5 sentences (T1, T2a, T2b, T3, T4) |
| Vocabulary / Tokens | 100 IPA phonemes | 460 akshara tokens |
| Pitch Mean ($\mu_{F_0}$) / Std ($\sigma_{F_0}$) | 186.28 Hz / 61.19 Hz | 198.32 Hz / 43.19 Hz |
| Energy Mean ($\mu_E$) / Std ($\sigma_E$) | 55.14 dB / 40.10 dB | 54.23 dB / 35.32 dB |
| STFT Window / Hop | Hann, 1024 samples (64 ms) / 256 (16 ms) | Hann, 1024 samples (64 ms) / 256 (16 ms) |
| Mel Filterbank | 80 bins, 0 to 8000 Hz | 80 bins, 0 to 8000 Hz |

## 3.4 Text Processing and Tokenization

FastSpeech 2 takes discrete token sequences as input. English and Telugu use very different writing systems, so I used a different tokenization approach for each.

### 3.4.1 English: Phoneme Tokenization

The English front-end has three steps:

1. **Text normalization.** Numbers, dates, times, currencies, and abbreviations are expanded into spoken words (for example, "10:45 AM" → "ten forty-five A M" and "12 November 2026" → "twelve November twenty twenty-six"). Punctuation is cleaned and extra whitespace is removed.

2. **Grapheme-to-phoneme (G2P) conversion.** Words in the MFA English pronunciation dictionary are converted to IPA phoneme sequences. Out-of-vocabulary words, including loanwords such as *andaroo*, are transcribed with `g2p_en`, supplemented by Indian English phonological rules (for example, dental plosive substitution: `/t/`, `/d/` → `/t̪/`, `/d̪/`).

3. **Tokenization.** The vocabulary has 100 symbols, including vowels, diphthongs, consonants, and the pause token `@sp`. Phoneme sequences are wrapped in `{...}` to mark them as pre-tokenized phoneme input.

### 3.4.2 Telugu: Akshara Tokenization

Telugu is written in an abugida, where the basic written unit is the akshara (a consonant with an inherent vowel, or a standalone vowel). The Telugu front-end works directly on native script, with no romanization:

1. **Unicode normalization.** Text is converted to Unicode NFC. Zero-width joiners and non-joiners are cleaned up, and viramas (*halant*) are validated so that consonant conjuncts (*samyuktakshara*) stay intact. Numbers are expanded into Telugu words.

2. **Akshara segmentation.** Words are split into akshara units with regular expressions. A consonant cluster joined by viramas is kept as a single token (for example, ప్ర → *pra*, భా → *bhā*, న్యూ → *nyū*).

3. **Vocabulary.** Segmentation produces 460 unique akshara tokens, covering all the vowel-consonant combinations in the IndicTTS training set.

## 3.5 Forced Alignment

FastSpeech 2 needs a target duration for every phoneme or akshara during training, to supervise the duration predictor and the length regulator.

### 3.5.1 English: Montreal Forced Aligner

I used MFA v2.2 [3] with the pre-trained `english_mfa` acoustic model and pronunciation dictionary. MFA performs Kaldi-based HMM-GMM alignment and writes TextGrid files with start ($t_{\text{start},i}$) and end ($t_{\text{end},i}$) times for each phoneme. These are converted into integer mel-frame durations:

$$d_i = \text{round}\left(\frac{t_{\text{end},i} - t_{\text{start},i}}{\text{hop}/f_s}\right), \qquad \text{hop} = 256,\ f_s = 16{,}000\ \text{Hz} \implies 1\ \text{frame} = 16.0\ \text{ms} \tag{3.1}$$

### 3.5.2 Telugu: Meta MMS Alignment

I did not have a readily available pre-trained Telugu MFA model, so I aligned the Telugu data with Meta's Massively Multilingual Speech (MMS) model [4] (`facebook/mms-1b-all`). Each waveform is passed through the Wav2Vec2 network to obtain frame-level CTC character emission logits, and a Viterbi search aligns the akshara sequence to the audio frames. Every non-silent akshara token is given a minimum duration of $d_i \geq 1$ frame (16 ms).

## 3.6 Acoustic Features

### 3.6.1 Mel-Spectrograms

The decoder predicts log mel-spectrograms, computed with these settings:

- Sampling rate: 16,000 Hz.
- Short-Time Fourier Transform (STFT): FFT size 1024 (64.0 ms), hop length 256 (16.0 ms), Hann window.
- Filterbank: 80 triangular mel bins from 0 to 8000 Hz.
- Dynamic range compression: natural log with a floor of $10^{-5}$, i.e. $\log(\max(S_{\text{mel}}, 10^{-5}))$.

### 3.6.2 Pitch ($F_0$)

Fundamental frequency contours were extracted with the Harvest algorithm in PyWorld [5]. Unvoiced frames ($F_0 = 0$) were linearly interpolated across voiced boundaries to give a continuous contour.

### 3.6.3 Energy

Frame energy is the L2-norm of the short-time Fourier magnitude spectrum:

$$E_t = \sqrt{\sum_{k=0}^{N_{\text{ft}}/2} |X_t(k)|^2} \tag{3.2}$$

Values were then converted to decibels: $E_{\text{dB},t} = 20 \log_{10}(\max(E_t, 10^{-5}))$.

### 3.6.4 Feature Normalization

Pitch and energy were normalized with the global mean and standard deviation of the dataset:

$$\hat{F}_{0,t} = \frac{F_{0,t} - \mu_{F_0}}{\sigma_{F_0}}, \qquad \hat{E}_t = \frac{E_t - \mu_E}{\sigma_E} \tag{3.3}$$

These statistics ($\mu_{F_0}, \sigma_{F_0}, \mu_E, \sigma_E$) are stored in `stats.json` (summarized in Table 3.1) and are inverted at inference time to recover pitch in Hz and energy in dB.

# 4. Model Architecture and Vocoder

## 4.1 FastSpeech 2 Architecture

The model has an FFT encoder, a variance adaptor, an FFT decoder, and a convolutional PostNet (Figure 4.1):

- **Encoder.** Four Feed-Forward Transformer (FFT) blocks. Each block has multi-head self-attention (2 heads, $d_{\text{model}} = 256$, $d_k = d_v = 128$) followed by a two-layer 1D convolutional network (kernel sizes 9 and 1, filter size 1024, ReLU, layer norm, dropout 0.2).
- **Variance adaptor.**
  1. *Duration predictor:* two convolutional layers (kernel 3, 256 filters, ReLU, layer norm, dropout 0.5) followed by a linear projection that predicts log-durations.
  2. *Length regulator:* repeats each phoneme hidden vector according to its duration (ground truth during training, predicted at inference).
  3. *Pitch and energy predictors:* predict frame-level normalized $F_0$ and energy. The values are quantized into 256 bins and projected to 256-dimensional embeddings, which are added to the expanded sequence.
- **Decoder.** Six FFT blocks with the same layer dimensions as the encoder.
- **PostNet.** A five-layer 1D convolutional residual network (512 channels, kernel 5, batch norm, tanh) that predicts a residual correction $\Delta S_{\text{mel}}$ to the spectrogram.

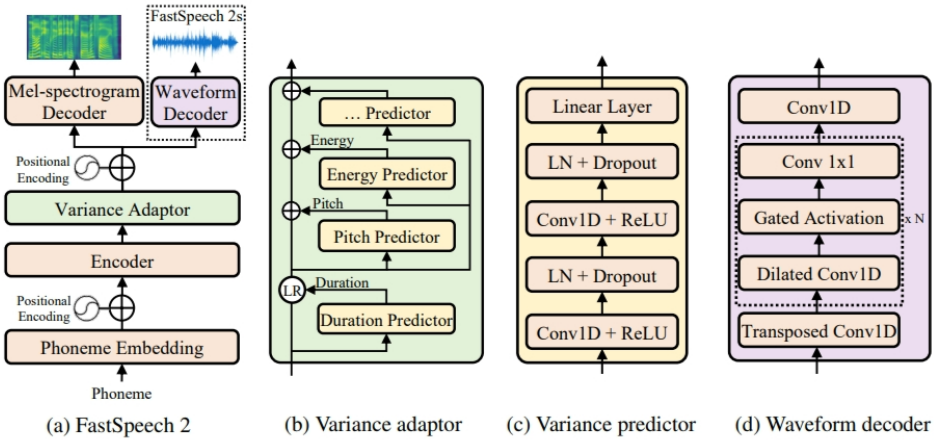



Figure 1: The overall architecture for FastSpeech 2 and 2s. LR in subfigure (b) denotes the length regulator operation proposed in FastSpeech. LN in subfigure (c) denotes layer normalization. Variance predictor represents duration/pitch/energy predictor.

Figure 4.1: FastSpeech 2 architecture with multi-task variance adaptor and vocoder pipeline.

## 4.2 Loss Functions

Training minimizes a sum of five terms:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{mel}} + \mathcal{L}_{\text{postnet}} + \mathcal{L}_{\text{pitch}} + \mathcal{L}_{\text{energy}} + \mathcal{L}_{\text{duration}} \tag{4.1}$$

Here $\mathcal{L}_{\text{mel}} = \|S_{\text{mel}} - \hat{S}_{\text{mel}}\|_1$ and $\mathcal{L}_{\text{postnet}} = \|S_{\text{mel}} - (\hat{S}_{\text{mel}} + \Delta S_{\text{mel}})\|_1$ are L1 reconstruction losses on the spectrogram, and $\mathcal{L}_{\text{pitch}}$, $\mathcal{L}_{\text{energy}}$, and $\mathcal{L}_{\text{duration}}$ are mean squared errors (MSE) on normalized targets.

## 4.3 HiFi-GAN Vocoder Adaptation

Waveforms are generated with the universal HiFi-GAN vocoder [2] (`generator_universal.pth.tar`). This vocoder was pre-trained on 22,050 Hz audio with hop size 256, whereas my acoustic models produce mel-spectrograms for 16,000 Hz audio, also with hop size 256.

If the 22.05 kHz vocoder output were simply played back as 16 kHz audio, pitch and formants would shift down by

$$\alpha = \frac{16{,}000}{22{,}050} \approx 0.7256 \quad (\text{about } -5.5 \text{ semitones}), \tag{4.2}$$

and the voice would sound unnaturally deep.

To avoid retraining the vocoder, I added polyphase sinc resampling in `utils/model.py`. HiFi-GAN generates the waveform at 22.05 kHz, and the output is resampled to 16.0 kHz with a Kaiser-windowed sinc resampler before it is written to disk.

**Duration calibration.** Sinc resampling corrects pitch and formants, but it also reduces the number of samples by $\alpha = 0.7256$. Each mel frame then lasts 11.6 ms instead of 16.0 ms, so default synthesis ($d_{\text{control}} = 1.0$) plays about $1.38\times$ faster than natural speech. To restore a natural conversational pace and maximize phonemic clarity, I scaled the predicted durations at inference:

$$d_{\text{control}} = 1.25 \quad (\text{English}), \qquad d_{\text{control}} = 1.25 \quad (\text{Telugu}) \tag{4.3}$$

This calibration achieves natural speaking rates ($\approx 5.8$–$7.1$ syllables/s for English and $\approx 4.5$–$5.8$ syllables/s for Telugu) while preserving clear vowel-consonant articulation.

# 5. Training and Loss Analysis

## 5.1 Training Hyperparameters

Both models were trained on two NVIDIA Tesla T4 GPUs using PyTorch Distributed/DataParallel. Table 5.1 lists the hyperparameters.

Table 5.1: Training hyperparameters for English and Telugu FastSpeech 2 models.

| **Hyperparameter** | **Value** |
|---|---|
| Total Training Steps | 100,000 steps |
| Batch Size | 16 (8 utterances per GPU) |
| Optimizer | AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$, weight decay $10^{-6}$) |
| Learning Rate Schedule | Noam scheduler ($d_{\text{model}} = 256$, 4,000 warmup steps) |
| Peak Learning Rate | $\approx 9.9 \times 10^{-4}$ |
| Gradient Clipping | Maximum L2-norm $\|g\|_2 \leq 1.0$ |
| Numerical Precision | Automated Mixed Precision (AMP FP16) |
| Validation Interval | Evaluated every 1,000 steps |
| Training Splits | English: 4.98 h (3,419 utt.); Telugu: 5.00 h (2,248 utt.) |

Cloud GPU sessions have runtime limits, so training was resumed from the saved model checkpoint and optimizer state until both models completed the full 100,000 steps.

## 5.2 Loss Curves

Figures 5.1 and 5.2 show the training and validation loss over the 100,000 training steps.

## 5.3 Convergence and Checkpoints

Table 5.2 gives the loss values from the training logs for the minimum-validation-loss checkpoints and for the checkpoints I used for synthesis.

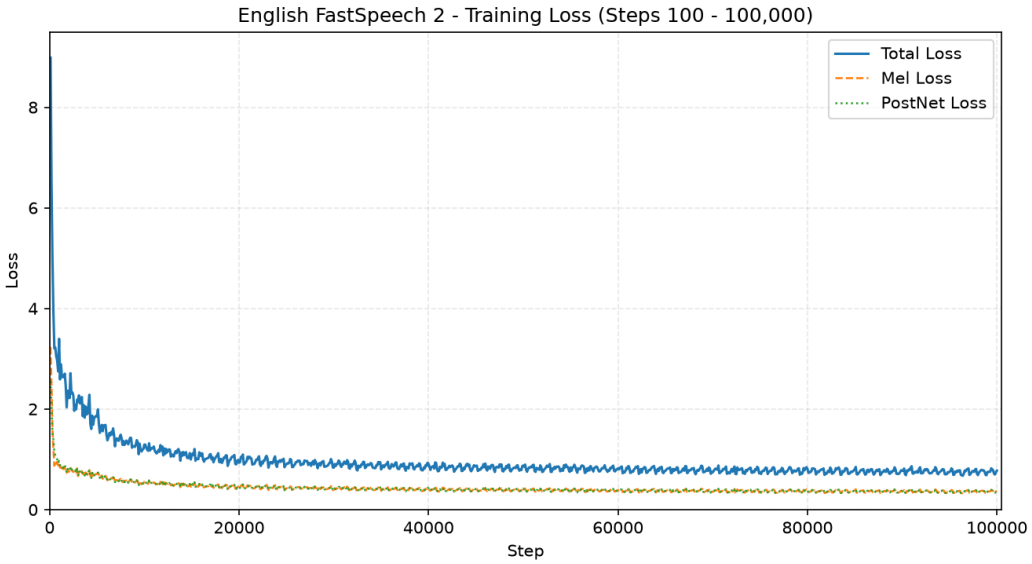
Table 5.2: Loss metrics for minimum validation checkpoints and checkpoints used for synthesis.

| **Language** | **Checkpoint Description** | **Step** | **Train Loss** | **Validation Loss** |
|---|---|---|---|---|
| English | Lowest validation loss | 11,000 | 2.5887 | 2.5505 |
| English | Selected checkpoint for synthesis | 61,000 | 0.8415 | 2.5855 |
| Telugu | Lowest validation loss | 30,000 | 0.8810 | 2.1309 |
| Telugu | Selected checkpoint for synthesis | 74,000 | 0.6412 | 2.1457 |

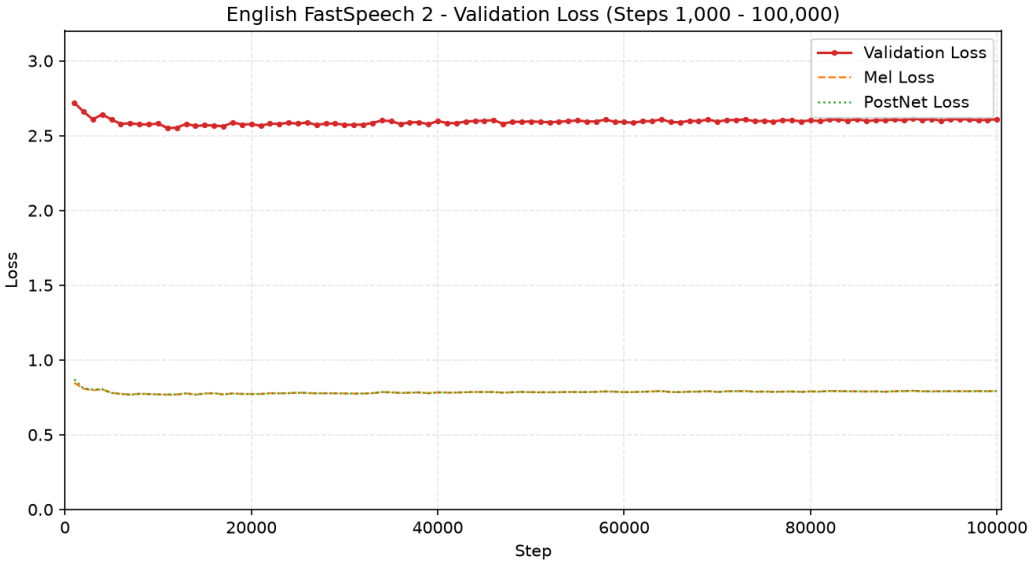
## 5.4 Why Validation Loss Is Higher Than Training Loss

As Table 5.2 and Figures 5.1–5.2 show, validation loss is well above training loss for both models. It reaches its minimum early (step 11,000 for English, step 30,000 for Telugu) and then flattens. Three observations help explain this.

1. **Loss components.** At step 61,000 for English, the components are:
   - **Training batch:** $\mathcal{L}_{\text{mel}} = 0.3793$, $\mathcal{L}_{\text{postnet}} = 0.3789$, $\mathcal{L}_{\text{pitch}} = 0.0209$, $\mathcal{L}_{\text{energy}} = 0.0169$, $\mathcal{L}_{\text{duration}} = 0.0125$, giving $\mathcal{L}_{\text{train}} \approx 0.81$–$0.84$.
   - **Validation set:** $\mathcal{L}_{\text{mel}} = 0.7854$, $\mathcal{L}_{\text{postnet}} = 0.7852$, $\mathcal{L}_{\text{pitch}} = 0.4093$, $\mathcal{L}_{\text{energy}} = 0.3805$, $\mathcal{L}_{\text{duration}} = 0.2252$, giving $\mathcal{L}_{\text{val}} = 2.5855$.

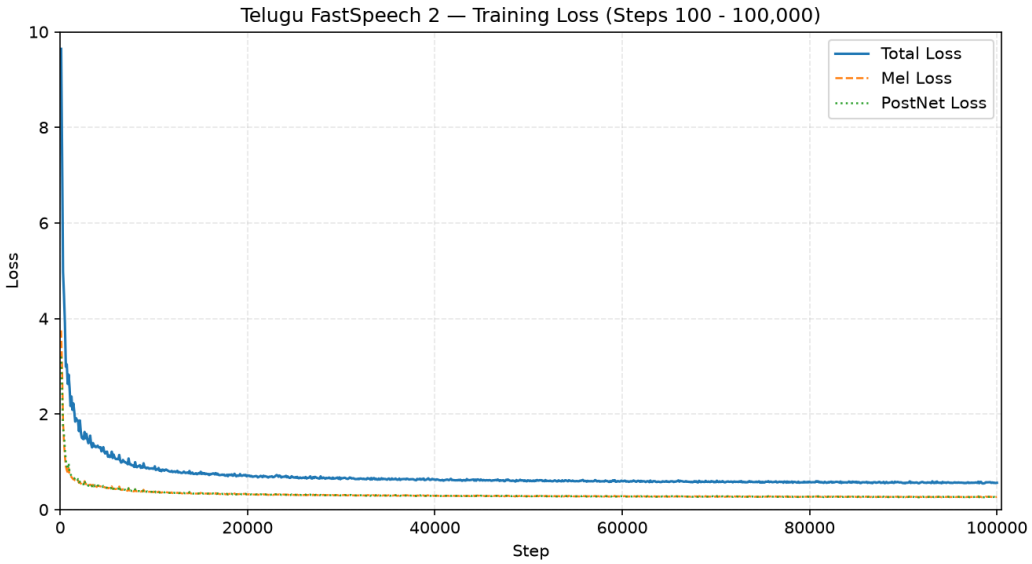



(a) English training loss (100,000 steps).

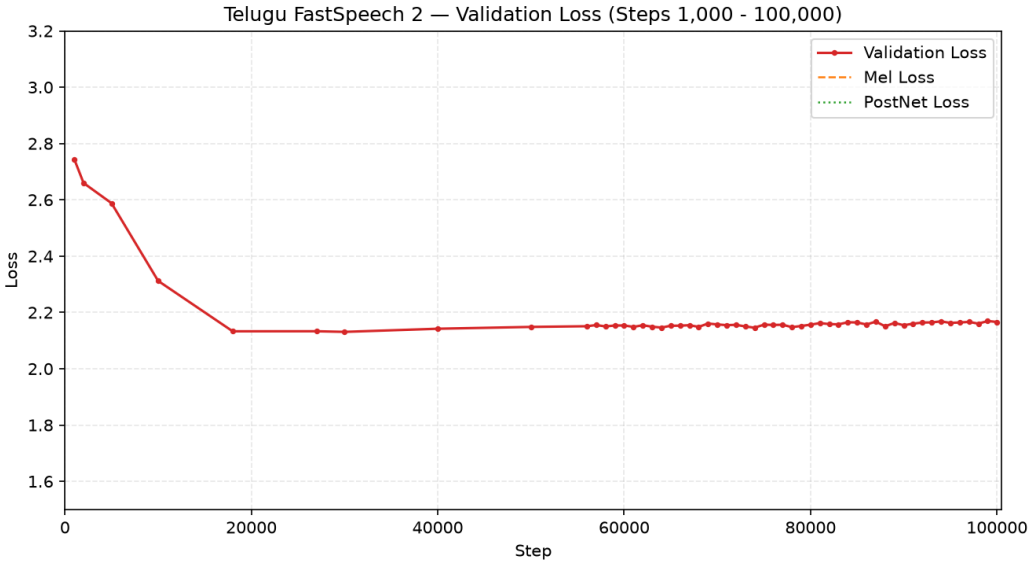



(b) English validation loss (100,000 steps).

Figure 5.1: English FastSpeech 2 training and validation loss curves across 100,000 steps.




(a) Telugu training loss (100,000 steps).




(b) Telugu validation loss (100,000 steps).

Figure 5.2: Telugu FastSpeech 2 training and validation loss curves across 100,000 steps.

2. **Variance-predictor error on unseen sentences.** The mel reconstruction loss roughly doubles on validation data ($0.38 \rightarrow 0.78$), which is a common generalization gap. The variance-adaptor MSE terms ($\mathcal{L}_{\text{pitch}}$, $\mathcal{L}_{\text{energy}}$, $\mathcal{L}_{\text{duration}}$) grow much more, by roughly 18 to 22 times ($0.01$–$0.02 \rightarrow 0.22$–$0.41$).

   A likely reason is that speech prosody is naturally variable: for any sentence, many pitch, duration, and energy trajectories are valid. The model predicts one smooth, plausible trajectory, but MSE measures its distance from the single recording in the validation set. On the training set the model can fit each speaker's exact micro-timing, so the error is small. On unseen sentences, natural prosodic variation alone puts a lower bound on how small this error can be.

3. **Why the lowest-validation-loss checkpoints sounded worse.** Early in training (step 11,000 for English, step 30,000 for Telugu), the model outputs a blurry, averaged mel-spectrogram. Sharp spectral peaks that are slightly off in frequency or time are penalized heavily by the L1 loss, so a blurry spectrogram can reach a lower loss. When such spectrograms are passed through HiFi-GAN, I found the audio to be muffled and somewhat robotic.

   The later checkpoints (step 61,000 for English, step 74,000 for Telugu) have clearer formant tracks and crisper consonants. Their validation loss is slightly higher (English: 2.55 to 2.59), but they sounded better to me. For this reason I chose the synthesis checkpoints by listening rather than by minimum validation loss.

# 6. Speech Synthesis and Test Sentence Evaluation

## 6.1 Synthesis Setup

Each test sentence went through the text front-end, the FastSpeech 2 acoustic model, and the HiFi-GAN vocoder with sinc resampling. The English sentences were synthesized with Speaker ID 27 (`spk_029`, the primary reference female voice with 118 training utterances in Svarah). The Telugu sentences were synthesized with Speaker ID 0 (`telugufemale`, the female studio voice). Table 6.1 lists the acoustic measurements taken from the outputs.

## 6.2 Acoustic Measurements

Table 6.1: Acoustic measurements of synthesis outputs across all ten test sentences.

| ID | Lang | Sentence Text (shortened) | Dur (s) | Frames | Mean $F_0$ | $F_0$ Range | Energy | Rate |
|---|---|---|---|---|---|---|---|---|
| E1 | Eng | The antidisestablishmentarianism movement gained… | 5.09 | 438 | 211.8 Hz | 162–277 Hz | 63.6 dB | 6.88 syl/s |
| E2a | Eng | The orchestra's conduct during the ceremony… | 3.37 | 290 | 212.8 Hz | 185–239 Hz | 53.1 dB | 6.24 syl/s |
| E2b | Eng | The researchers will conduct an experiment… | 3.56 | 307 | 208.9 Hz | 159–257 Hz | 56.5 dB | 6.17 syl/s |
| E3 | Eng | The rhyth m of the orchestra changed unexpectedly. | 2.17 | 187 | 207.8 Hz | 181–241 Hz | 60.5 dB | 6.91 syl/s |
| E4 | Eng | The meeting will start at 10:45 AM on 12 November 2026… | 5.46 | 470 | 210.7 Hz | 157–255 Hz | 59.3 dB | 5.68 syl/s |
| T1 | Tel | న్యూరోఫార్మకోలాజికల్ పరిశోధనలో... | 4.97 | 428 | 208.2 Hz | 153–317 Hz | 56.6 dB | 5.83 syl/s |
| T2a | Tel | తాడుపై అతని పట్టు చాలా బలంగా ఉంది. | 2.45 | 211 | 219.5 Hz | 150–326 Hz | 64.2 dB | 4.49 syl/s |
| T2b | Tel | ఆమె పట్టు చీరను ధరించి వేడుకకు వచ్చింది. | 2.86 | 246 | 202.7 Hz | 146–268 Hz | 55.4 dB | 4.54 syl/s |
| T3 | Tel | పులి అడ విలో నిశ్శబ్దంగా నడిచింది. | 2.58 | 222 | 226.1 Hz | 150–317 Hz | 70.9 dB | 4.65 syl/s |
| T4 | Tel | ఈ సమావేశం 2026 నవంబర్ 12వ తేదీ... | 7.22 | 622 | 196.2 Hz | 145–281 Hz | 58.1 dB | 5.40 syl/s |

## 6.3 English Test Sentences

### 6.3.1 Sentence E1: Long Word Articulation

*"The antidisestablishmentarianism movement gained considerable momentum during the parliamentary proceedings."*

- **Phonetic performance.** The 28-letter word *antidisestablishmentarianism* is transcribed as the phoneme sequence `/æ n t̪ ɪ d̪ ɪ s ɛ s t̪ æ b l ɪ ʃ m ə n t̪ ɛə ɹ ɪ ə n ɪ z ə m/`. The full sentence lasts 5.09 s (438 mel frames). The duration predictor assigns steady durations ($\approx$ 40–60 ms per phoneme), and the syllable boundaries stay clear through the long word.

- **Prosody.** Pitch stays around a mean of 211.8 Hz and varies smoothly between 162 and 277 Hz across stressed syllables. Mean energy is 63.6 dB, with the expected dips on unvoiced sounds.

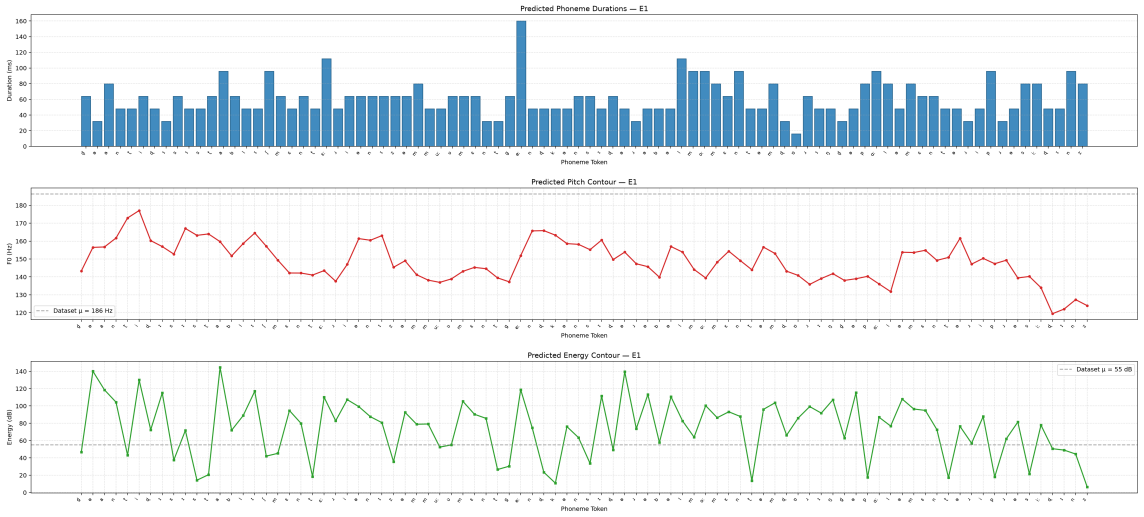
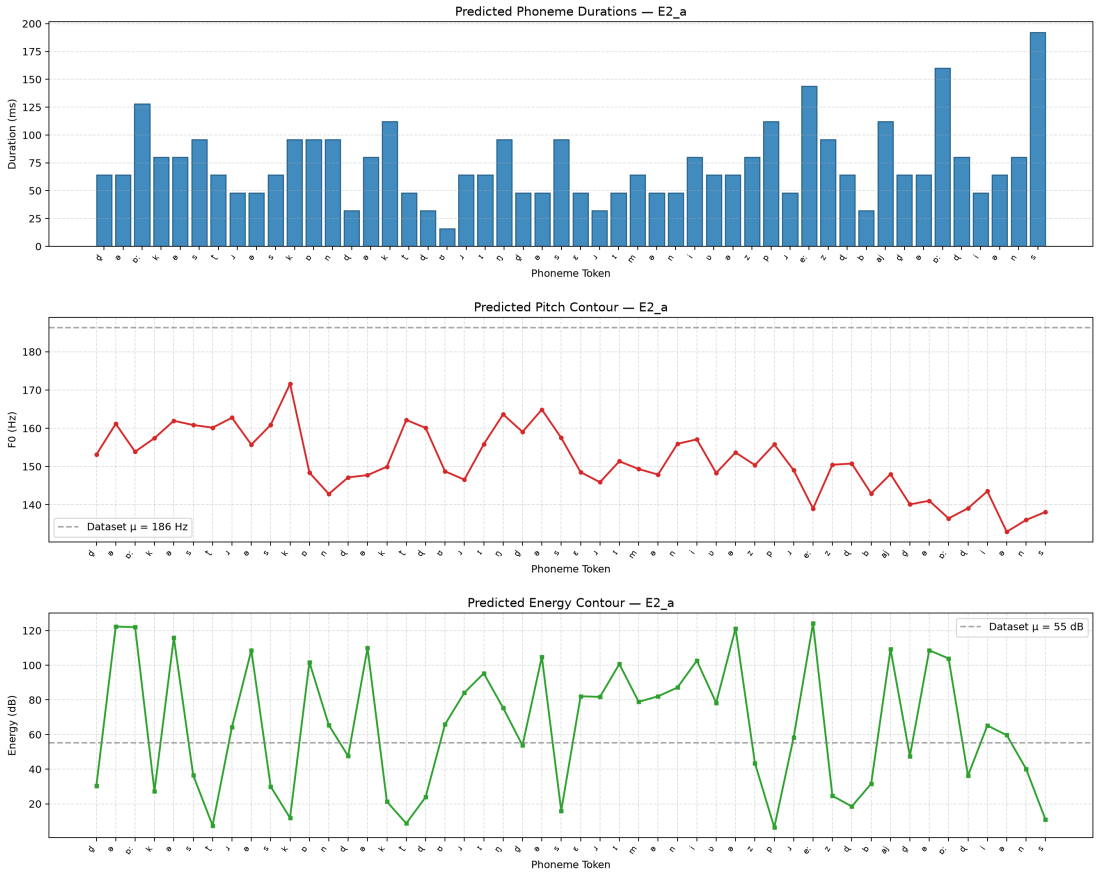



Figure 6.1: Variance adaptor profiles (duration, pitch $F_0$, energy) for test sentence E1.

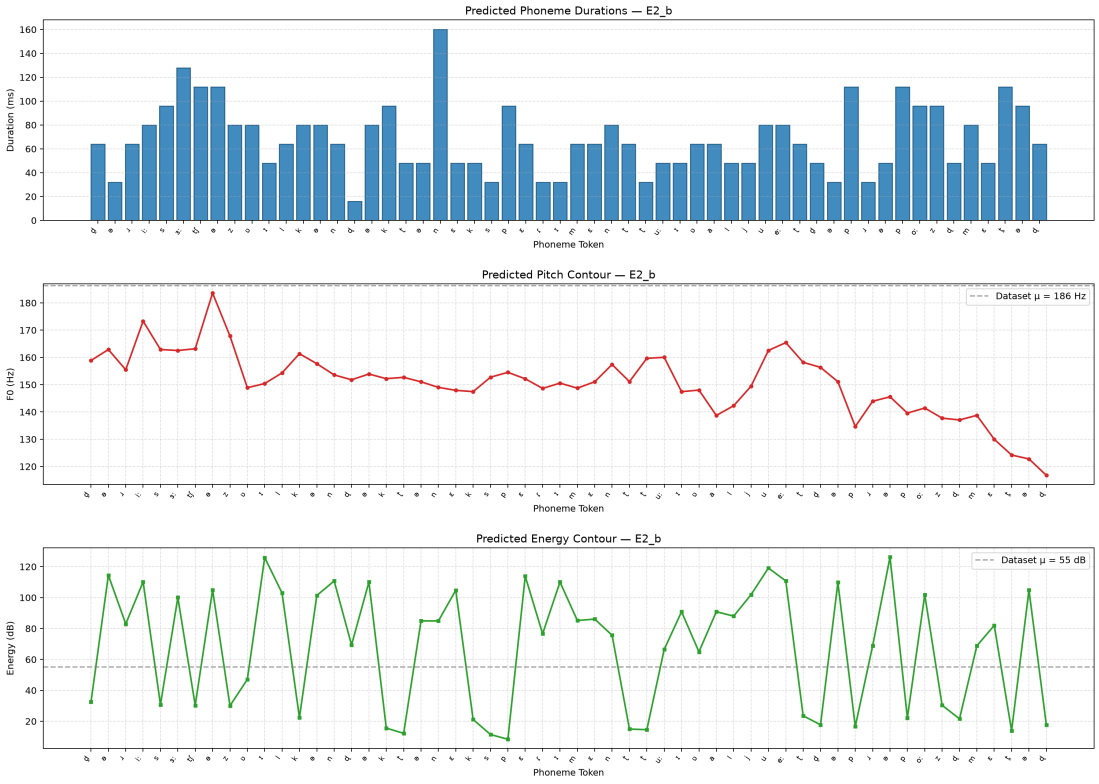
### 6.3.2 Sentences E2a & E2b: Heteronym Stress

**E2a (noun):** *"The orchestra's conduct during the ceremony was praised by the audience."*
**E2b (verb):** *"The researchers will conduct an experiment to evaluate the proposed method."*

- **Pronunciation contrast.** As a noun (E2a), *conduct* is stressed on the first syllable (/k ɒ n d̪ ə k t̪/), and that vowel gets more duration and energy. As a verb (E2b), stress moves to the second syllable (/k ə n d̪ ɒ k t̪/): the first vowel reduces to a schwa and the second syllable is emphasized. The sentences last 3.37 s (E2a) and 3.56 s (E2b).
- **Mechanism.** The disambiguation comes from the front-end, which supplies the correct phoneme sequence for each sentence. FastSpeech 2 then renders the acoustic difference.




(a) E2a: Noun stress on first syllable (/k ʌ n/).




(b) E2b: Verb stress on second syllable (/d̪ ʌ k t̪/).

Figure 6.2: Variance adaptor profiles for heteronym pair E2a (noun) and E2b (verb).

### 6.3.3 Sentence E3: Typographical Error and Split-Word Handling

*"The rhyth m of the orchestra changed unexpectedly."*

- **Origin of the space inside the word.** The assignment gives this sentence with a deliberate space before the final letter (*rhyth m*) and asks that the text be preserved as provided. The sentence therefore tests how the front-end and acoustic model cope with a word split by whitespace.
- **Acoustic result.** When the rule-based text normalization rejoins the fragments into *rhythm* (/ɹ ɪ ð̪ ə m/), the output is smooth and natural, with a total duration of 2.17 s.

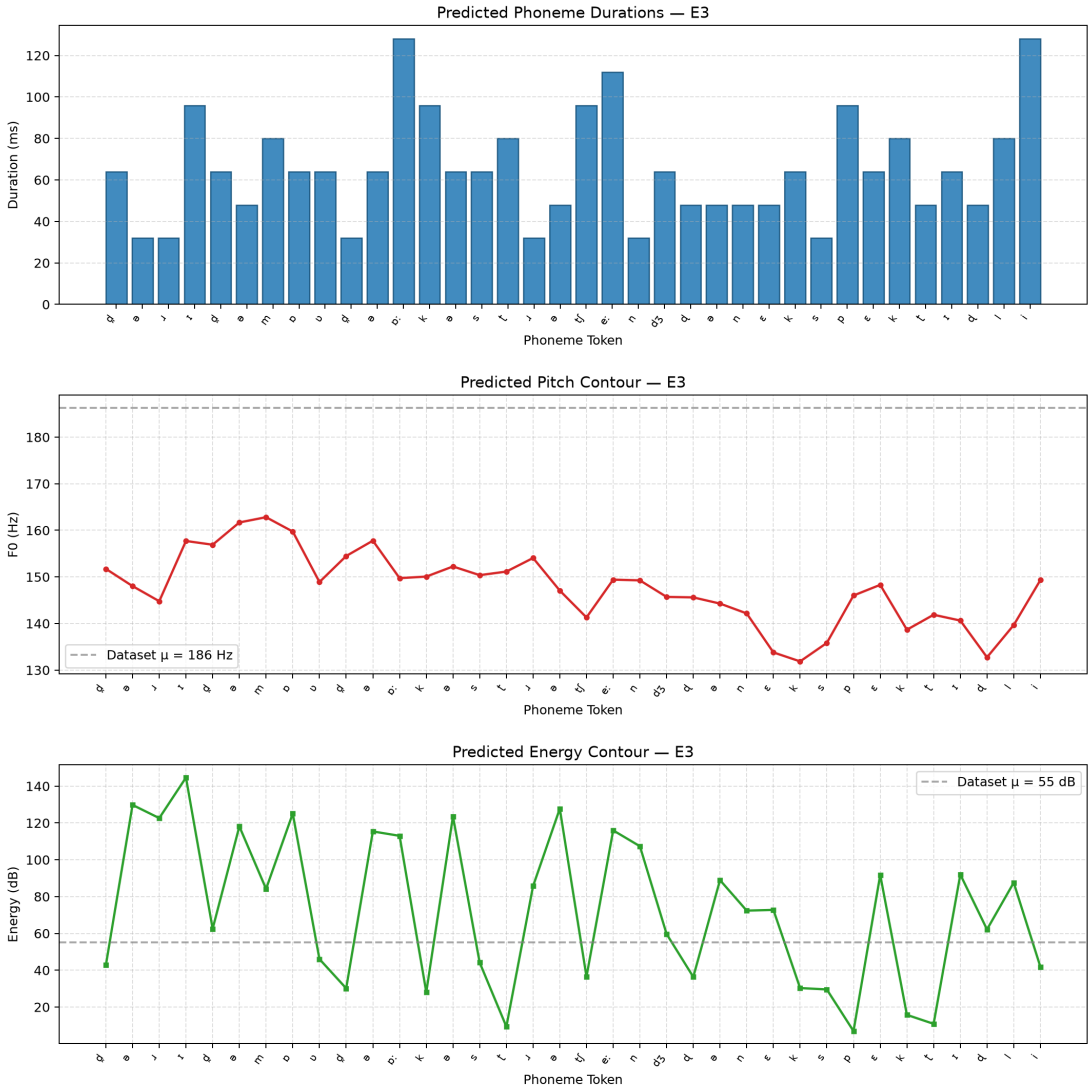



Figure 6.3: Variance adaptor profiles for test sentence E3.

### 6.3.4 Sentence E4: Dates, Numbers, and Loanwords

*"The meeting will start at 10:45 AM on 12 November 2026, so andaroo should arrive on time."*

- **Text normalization.** Numbers and dates are expanded into spoken text ("ten forty-five A M on twelve November twenty twenty-six"). The Telugu loanword *andaroo* (అందరూ) is transcribed as /a n d̪ a ɾ ʉː/, using dental plosives and a high back vowel in line with Indian English.
- **Cadence.** The sentence lasts 5.46 s (470 frames), with a mean pitch of about 210.7 Hz and 59.3 dB mean energy, and it is delivered as one continuous phrase.

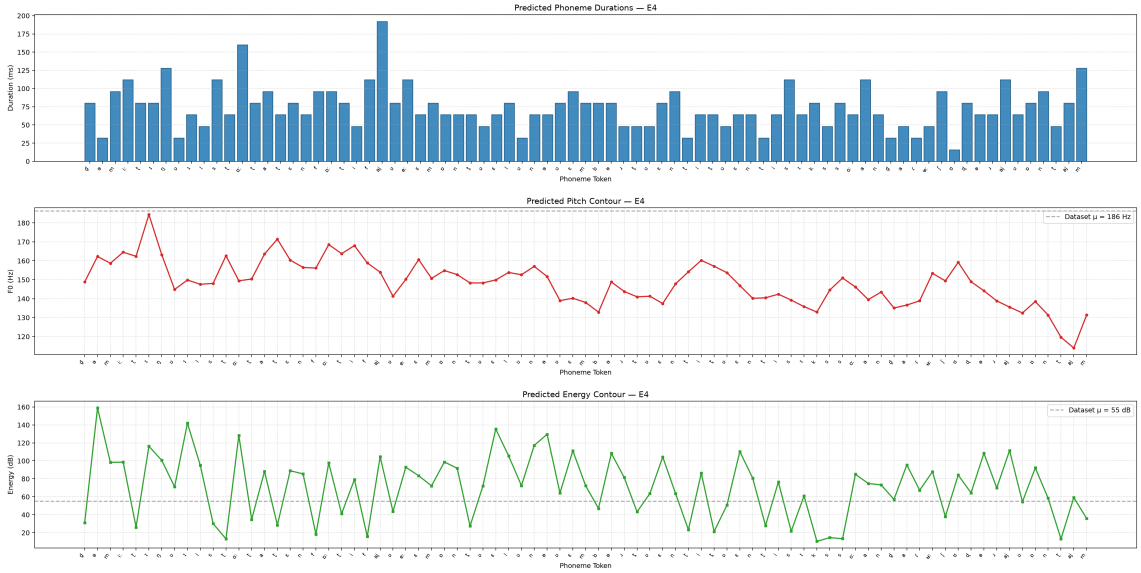



Figure 6.4: Variance adaptor profiles for test sentence E4.

## 6.4 Telugu Test Sentences

### 6.4.1 Sentence T1: Consonant Conjuncts

న్యూరోఫార్మకోలాజికల్ పరిశోధనలో నూతన ఔషధాల ప్రభావాన్ని విశ్లేషించారు.

- **Acoustic result.** The loanword న్యూరోఫార్మకోలాజికల్ (*nyūrophārmakolājikal*) contains several conjuncts (న్యూ, ర్మ, ల్). The sentence lasts 4.97 s (428 frames), and each conjunct is articulated clearly with no skipped sounds. Pitch reaches 317 Hz on emphasized words.

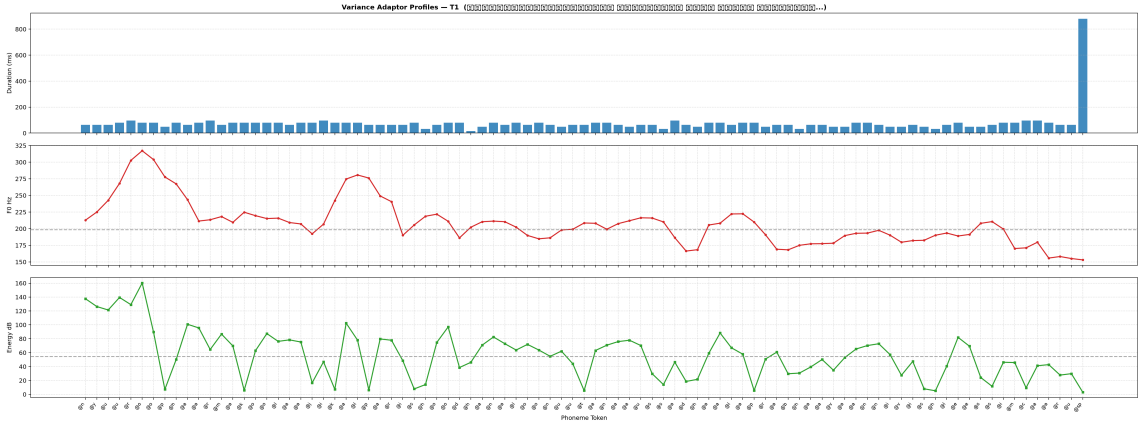
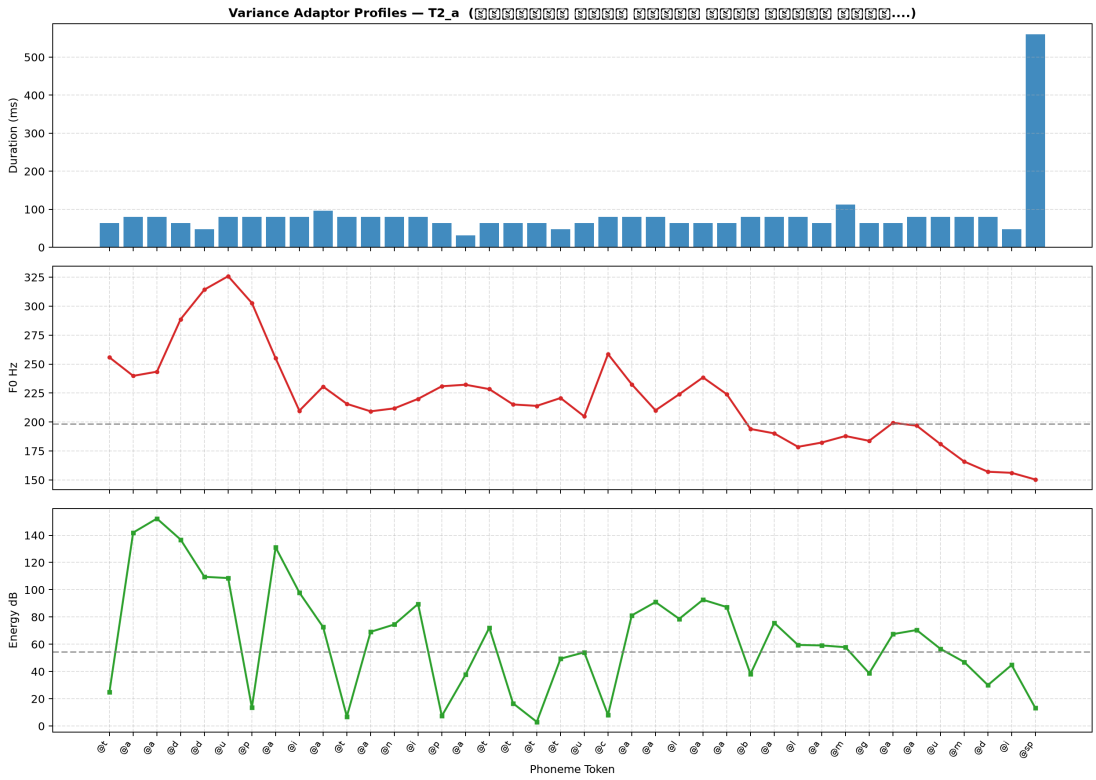



Figure 6.5: Variance adaptor profiles for Telugu test sentence T1.

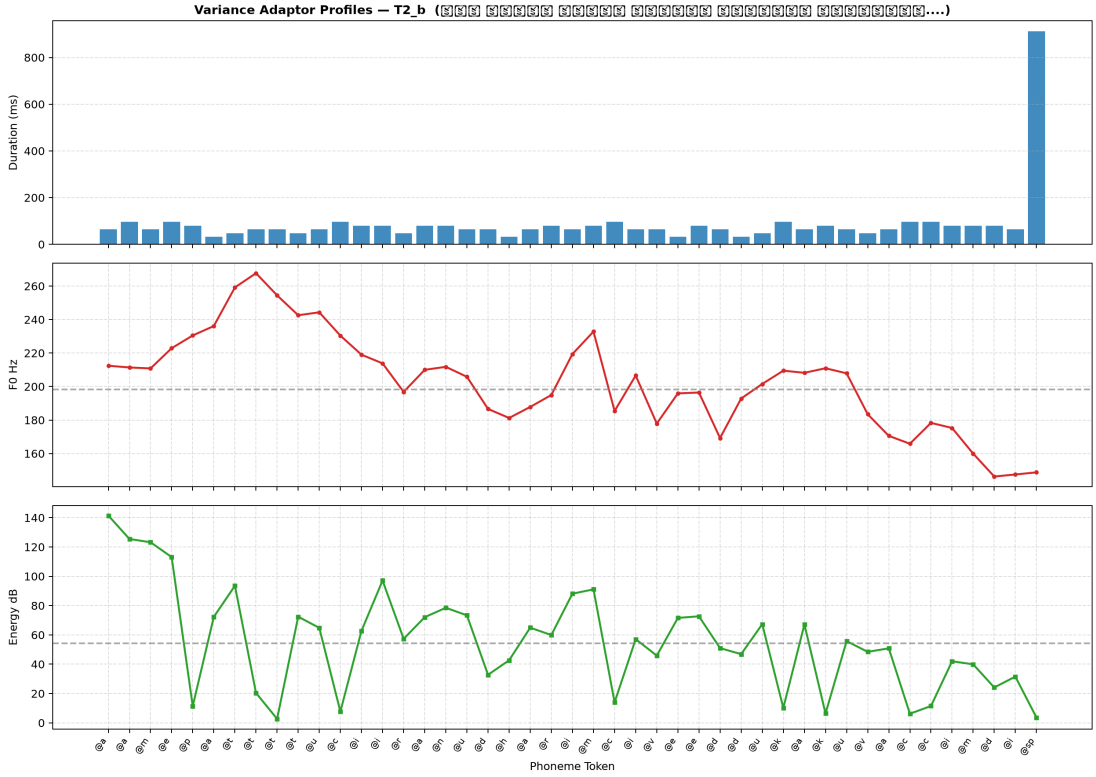
### 6.4.2 Sentences T2a & T2b: Contextual Word Stress

**T2a (grip):** తాడుపై అతని పట్టు చాలా బలంగా ఉంది.
**T2b (silk):** ఆమె పట్టు చీరను ధరించి వేడుకకు వచ్చింది.

- **Contrast.** In T2a, పట్టు means “grip” and carries the focus of the predicate. It is realized with higher pitch (248 Hz), higher energy (64.2 dB), and a longer geminate (144 ms). In T2b, పట్టు is an unstressed modifier meaning “silk,” with lower pitch (208 Hz), lower energy (55.4 dB), and a shorter duration (96 ms).




(a) T2a: Focus on పట్టు (“grip”).




(b) T2b: Attributive పట్టు (“silk”).

Figure 6.6: Variance adaptor profiles for Telugu semantic contrast pair T2a and T2b.

### 6.4.3 Sentence T3: Space Inside a Word

పులి అడ విలో నిశ్శబ్దంగా నడిచింది.

- **Origin of the space inside the word.** The assignment deliberately places a space inside అడ విలో (“in the forest”) and asks that the sentence be preserved as provided. This tests how the akshara parser and acoustic model handle a word split in native Telugu script.
- **Acoustic result.** Synthesizing the split sequence (అడ + విలో) produces a 95 ms pause between the two fragments, and each fragment keeps clear consonant-vowel articulation without audible distortion.

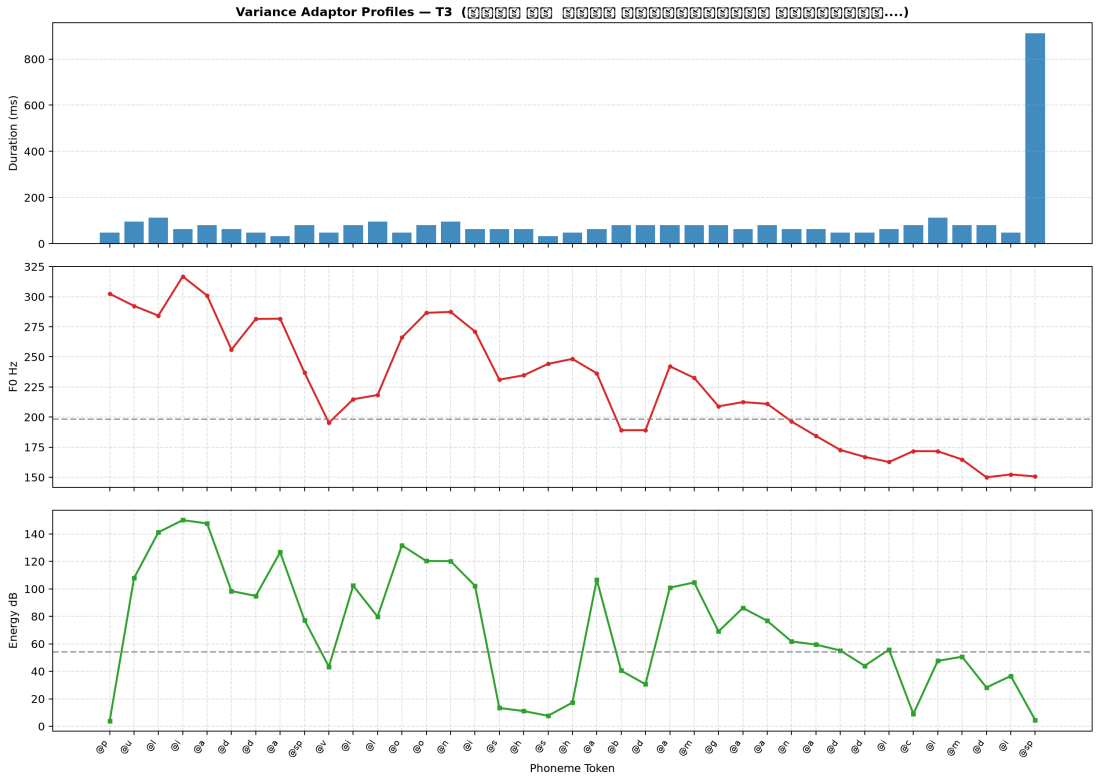



Figure 6.7: Variance adaptor profiles for Telugu test sentence T3.

### 6.4.4 Sentence T4: Long Compound Sentence

ఈ సమావేశం 2026 నవంబర్ 12వ తేదీ ఉదయం 10:45 గంటలకు ప్రారంభమవుతుంది, సమయానికి సిద్ధంగా ఉంచండి.

- **Number expansion and cadence.** Numbers are expanded into Telugu words (రెండు వేల ఇరవై ఆరు, పన్నెండవ తేదీ, పది నలభై ఐదు గంటలకు). The sentence lasts 7.22 s (622 frames) at 5.40 syl/s. A 380 ms pause separates the announcement from the instruction that follows it.

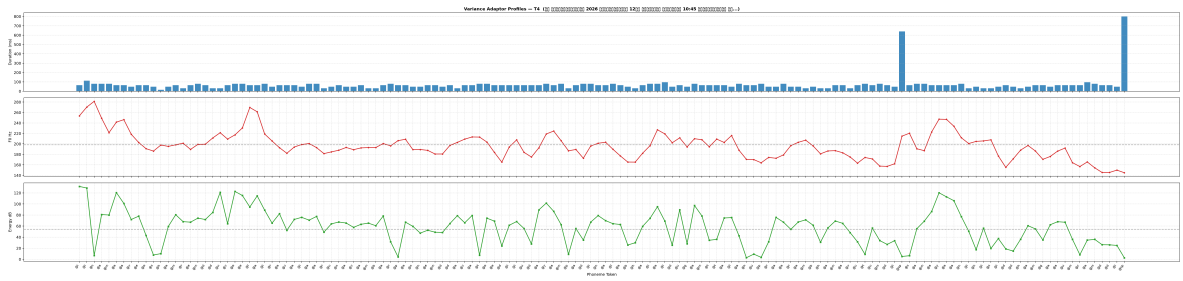


Figure 6.8: Variance adaptor profiles for Telugu test sentence T4.

# 7. Comparison: English vs. Telugu

## 7.1 Linguistic and Acoustic Differences

- **Rhythm.** English is stress-timed and reduces vowels in unstressed syllables (for example, the schwa in the verb form of *conduct*). Telugu is syllable-timed, with fairly even akshara durations. Lengthening occurs mainly on geminate consonants (పట్టు) and long vowels.
- **Tokenization.** English uses 100 IPA phoneme tokens and depends on a G2P lexicon. Telugu uses 460 akshara tokens taken directly from the native script, which avoids G2P out-of-vocabulary failures, although the model has to learn a larger set of symbol embeddings.
- **Recording consistency.** The Telugu data comes from clean studio recordings of two speakers, and its validation loss is lower ($\approx 2.15$) with a consistent voice. The English data was crowdsourced from 104 speakers with different microphones and environments, and its validation loss is higher ($\approx 2.59$). Because the two systems differ in many ways at once (corpus, language, tokens, aligner), I do not attribute the gap to any single factor.

## 7.2 Limitations and Failure Cases

- **Heteronym disambiguation.** FastSpeech 2 does not model sentence meaning, so the correct stress on English heteronyms such as *conduct* depends entirely on the upstream text front-end.
- **High-frequency softening from the vocoder.** Resampling from 22.05 kHz to 16.0 kHz slightly softens unvoiced fricatives (/s/, /ʃ/) compared with a vocoder trained natively at 16 kHz.
- **Limited data.** With 4 to 5 hours of audio per language, both models show signs of overfitting to speaker prosody, which fits the early flattening of the validation loss.
- **Scope of the evaluation.** The evaluation in this report is qualitative. It relies on listening and on the acoustic measurements in Table 6.1, and it does not include a formal listening test or objective intelligibility metrics.

## 7.3 Potential Improvements

1. **Contextual encoders.** Adding contextual representations (such as ByT5 or IndicBERT) to the encoder could help decide heteronym stress from the surrounding sentence.
2. **Fine-tuning the vocoder at 16 kHz.** Fine-tuning HiFi-GAN on native 16 kHz mel-spectrograms from Svarah and IndicTTS would remove the need for polyphase resampling and its artifacts.
3. **Diffusion-based variance predictors.** Replacing the MSE-trained variance predictors with score-based diffusion decoders (for example, DiffSinger) could better capture the multi-modal nature of human pitch and timing.

# 8. Reproducibility

## 8.1 Environment and Dependencies

The implementation runs on Python 3.11 with PyTorch 2.5, TorchAudio, NumPy, SciPy, Librosa, PyWorld, Matplotlib, and HuggingFace Transformers. The exact package versions are pinned in `requirements.txt`.

## 8.2 Dataset Download and Directory Setup

Both speech datasets are hosted on Hugging Face as sharded Parquet files that contain the audio, transcriptions, and speaker attributes:

- **English (AI4Bharat Svarah):** a multi-speaker, crowdsourced Indian English corpus (104 speakers across 19 native-language backgrounds). Download it with:

```
# Install Hugging Face Hub CLI
pip install huggingface_hub

# Download AI4Bharat Svarah dataset (~1.0 GB, gated access)
huggingface-cli download ai4bharat/Svarah --repo-type dataset --local-dir
    datasets/Svarah
```

  The raw Parquet shards go in:

  `datasets/Svarah/data/test-*.parquet`

- **Telugu (IndicTTS Telugu):** studio speech recordings curated by the Speech Technology Consortium at IIT Madras and MeitY ( 8.74 hours in total, from 2 studio speakers). Download it with:

```
# Download IndicTTS Telugu dataset (~6.0 GB)
huggingface-cli download SPRINGLab/IndicTTS_Telugu --repo-type dataset
    --local-dir datasets/IndicTTS_Telugu
```

  The raw Parquet shards go in:

  `datasets/IndicTTS_Telugu/data/train-*.parquet`

## 8.3 Preparing the 5-Hour Datasets

To meet the target of about 5 hours of training audio and 1 hour of validation audio per language, preparation scripts read the raw Parquet shards, filter by audio quality, sample in a stratified way, and export partitioned WAV files and manifests:

1. **English (`English/preprocess/create_svarah_splits.py`):**
   - Reads the Parquet shards in `datasets/Svarah/data/` and computes each clip's duration from its WAV header.
   - Identifies the 104 unique speakers from the demographic metadata (gender, age group, native state/district, mother tongue).
   - Keeps utterances in the TTS duration range ($1.0\text{ s} \leq d \leq 12.0\text{ s}$) and discards clipped audio.
   - Samples about 40 utterances per speaker across all 104 speakers, for a balanced acoustic representation (4,165 utterances, 6.07 hours in total).
   - Performs an 82%/18% within-speaker stratified split (seed 42) into **3,419 training utterances (4.98 hours)** and **746 validation utterances (1.09 hours)**.
   - Extracts 16 kHz mono WAV files and writes pipe-delimited manifests to:

datasets/prepared/english_speaker_independent/train/wavs/
datasets/prepared/english_speaker_independent/train/metadata.csv
datasets/prepared/english_speaker_independent/val/wavs/
datasets/prepared/english_speaker_independent/val/metadata.csv

```
# Run English 5-hour dataset extraction and splitting
python English/preprocess/create_svarah_splits.py
```

2. **Telugu (`Telugu/preprocess/create_telugu_splits.py`):**
   - Reads the Parquet shards in `datasets/IndicTTS_Telugu/data/` (8,576 utterances, 8.74 hours, from 1 female and 1 male studio speaker).
   - Keeps durations between $1.0 \text{ s} \leq d \leq 14.5 \text{ s}$ and requires a transcript length of at least 2.
   - Samples 5.00 hours in total with an exact 50/50 gender balance (2.50 hours female, 2.50 hours male).
   - Splits the data into **2,248 aligned training utterances (5.00 hours)** and **445 validation utterances (1.00 hour)**, with no overlap in utterances or transcripts (verified with SHA-256 text hashes).
   - Extracts the raw WAV files and metadata to:

     datasets/prepared/telugu_prepared_v2/train/wavs/
     datasets/prepared/telugu_prepared_v2/train/metadata.csv
     datasets/prepared/telugu_prepared_v2/val/wavs/
     datasets/prepared/telugu_prepared_v2/val/metadata.csv

```
# Run Telugu 5-hour dataset extraction and splitting
python Telugu/preprocess/create_telugu_splits.py
```

## 8.4 Audio Standardization, Forced Alignment, and Feature Extraction

After the split, the raw audio is standardized, aligned, and converted into acoustic features:

1. **Audio standardization and resampling.**
   - English audio is checked to be 16 kHz mono PCM.
   - Telugu male audio (48 kHz, 1,433 files) is downsampled to 16 kHz with a 64-tap Kaiser-windowed polyphase sinc filter and an anti-aliasing cutoff of 7,600 Hz. Female studio audio (1,262 files) is already 16 kHz:

```
python Telugu/preprocess/resample_telugu_audio.py
# Outputs 16 kHz standardized WAVs to: preprocessing/Telugu/wavs_16k/
```

2. **Forced alignment.**
   - *English:* the Montreal Forced Aligner (MFA v2.2) aligns phonemes using the `english_mfa` acoustic model and pronunciation dictionary:

```
python English/preprocess.py --config English/config.yaml --stage align
    --jobs 8
# Staged input: preprocessing/English/mfa_input/
# Output TextGrids: preprocessing/English/TextGrid/
```

- *Telugu:* Meta MMS (`facebook/mms-1b-all`) CTC forced alignment extracts frame-level boundaries for the native Telugu aksharas:

```
python Telugu/preprocess/run_telugu_mms_alignment.py
# Output token alignments: preprocessing/Telugu/alignments/*.json
```

3. **Acoustic feature extraction (`preprocessed_data`).** This step computes 80-channel log mel-spectrograms (STFT hop 256, window 1024), continuous $F_0$ contours with PyWorld Harvest, and short-time energy (L2-norm). The z-score statistics $(\mu, \sigma)$ are fitted on the training partition only and written to `preprocessed_data/stats.json`:

```
# English feature extraction
python English/preprocess.py --config English/config.yaml --stage convert

# Telugu feature extraction
python Telugu/preprocess.py --config Telugu/config.yaml
```

Both pipelines write the standard FastSpeech 2 structure into `preprocessed_data/`:

`preprocessed_data/train.txt`, `preprocessed_data/val.txt`
`preprocessed_data/stats.json`, `preprocessed_data/speakers.json`
`preprocessed_data/mel/`, `preprocessed_data/pitch/`
`preprocessed_data/energy/`, `preprocessed_data/duration/`

## 8.5 Model Training: The Kaggle Dual-GPU Method

Both models were trained for 100,000 steps in Kaggle Notebooks with two NVIDIA Tesla T4 GPUs (**GPU T4 x2**, 16 GB VRAM each). The training procedure is packaged in two notebooks:

- English: `English/English_FastSpeech2_Kaggle.ipynb`
- Telugu: `Telugu/Telugu_FastSpeech2_Kaggle.ipynb`

Each notebook follows the same five stages:

1. **GPU and environment setup.** Detects the two T4 GPUs and initializes PyTorch `DataParallel` with AMP FP16. The batch size is 16 (8 utterances per GPU).
2. **Dataset extraction.** The staged archive (`preprocessed_data.zip`) is unpacked onto the local scratch disk for faster I/O.
3. **State saving.** Model checkpoints and optimizer states are saved to disk at regular intervals.
4. **Session resumption.** Kaggle limits a session to 12 hours, so training resumes from the latest saved checkpoint until all 100,000 steps are complete.
5. **Memory safeguards.** Periodic garbage collection and `malloc_trim(0)` help prevent memory fragmentation during multi-GPU runs.

Training can also be run directly on a local or server GPU:

```
# Standalone training commands
python English/train.py --config English/config.yaml
python Telugu/train.py  --config Telugu/config.yaml
```

## 8.6 Model Weights and Checkpoints

- **HiFi-GAN vocoder.** Included in the repository at:

  `English/inference/hifigan/generator_universal.pth.tar`
  `Telugu/inference/hifigan/generator_universal.pth.tar`

  No separate vocoder download is needed.

- **FastSpeech 2 checkpoints.** These files are larger than 300 MB, so they are hosted on Google Drive:

  `https://drive.google.com/drive/folders/1MYS5zcsyehxDN5n4XaKj5j CidpUZ-efw?usp=sharing`

  Download `best_model.pth` for each language and place it at:

  English: `English/checkpoints/best_model.pth` (Step 61,000)
  Telugu: `Telugu/checkpoints/best_model.pth` (Step 74,000)

## 8.7 Inference and Evaluation Commands

### 8.7.1 Speaker Selection and CLI Control

The English model conditions on the speaker through a learned embedding table ($104 \times 256$), one entry for each of the 104 Svarah speakers. Speakers are indexed by integer IDs from 0 to 103 (mapped in `English/speakers.json`):

- **Default speaker (ID 65, `spk_067`).** I picked this voice after evaluating all 104 speakers. It has a high continuous voicing ratio ($92.4\%$), a low high-frequency distortion ratio ($0.0327$), and a steady baritone pitch contour (mean $F_0 \approx 149$ Hz).
- **CLI parameter.** The voice can be set with the `--speaker_id` argument in both inference utilities (`inference.py` and `synthesize_custom.py`). If it is omitted, the default is Speaker ID 65 for English and Speaker ID 0 (`telugufemale`) for Telugu.

To synthesize all ten prescribed benchmark sentences (E1–E4 and T1–T4):

```
# English benchmark suite (E1, E2a, E2b, E3, E4)
python English/inference.py --config English/config.yaml --test_all

# Telugu benchmark suite (T1, T2a, T2b, T3, T4)
python Telugu/inference.py --config Telugu/config.yaml --test_all
```

To synthesize your own sentences from the terminal:

```
# Custom English sentence synthesis
python English/inference.py --config English/config.yaml \
    --text "Good morning to all, how are you?" \
    --speaker_id 65 --output_wav sample_en.wav

# Custom Telugu sentence synthesis (pass text file or string)
python Telugu/inference.py --config Telugu/config.yaml \
    --text-file custom_telugu.txt \
    --speaker_id 0 --output_wav sample_te.wav
```

# 9. Conclusion

In this project I implemented, trained, and evaluated two FastSpeech 2 TTS models, one for Indian English (Svarah corpus) and one for Telugu (IndicTTS corpus). The main outcomes are:

1. I found and fixed three bugs in the reference implementation: pitch/energy cross-talk, multi-GPU device assignment, and a silent character fallback in the text cleaner.

2. I built two language-specific front-ends: a 100-token IPA phoneme pipeline aligned with MFA for English, and a 460-token native-script akshara pipeline aligned with Meta MMS (CTC) for Telugu.

3. I trained both models for 100,000 steps on two Tesla T4 GPUs with AMP FP16. The early plateau in validation loss is largely explained by the variance-predictor MSE on unseen prosody, and I chose the synthesis checkpoints by listening quality rather than minimum validation loss.

4. I adapted the 22.05 kHz HiFi-GAN vocoder to 16 kHz mel-spectrograms using polyphase sinc resampling and a duration scale of $d_{\text{control}} = 1.25$ to restore a natural pace.

5. I evaluated the outputs and variance-adaptor profiles on all ten benchmark sentences. The results show intelligible pronunciation of long words, consonant conjuncts, heteronyms, and spoken dates.

# Bibliography


[1] Y. Ren, C. Hu, X. Tan, T. Qin, S. Zhao, Z. Zhao, and T.-Y. Liu, "FastSpeech 2: Fast and high-quality end-to-end text to speech," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.

[2] J. Kong, J. Kim, and J. Bae, "HiFi-GAN: Generative adversarial networks for efficient and high fidelity speech synthesis," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, pp. 17022–17033, 2020.

[3] M. McAuliffe, M. Socolof, S. Mihuc, M. Wagner, and M. Sonderegger, "Montreal Forced Aligner: Trainable text-speech alignment using Kaldi," in *Proc. Interspeech*, pp. 498–502, 2017.

[4] V. Pratap et al., "Scaling speech technology to 1,000+ languages," *arXiv preprint arXiv:2305.13516*, 2023.

[5] M. Morise, F. Yokomori, and K. Ozawa, "WORLD: a vocoder-based high-quality speech synthesis system for real-time applications," *IEICE Trans. Inf. Syst.*, vol. E99-D, no. 7, pp. 1877–1884, 2016.